ANTHROPIC

Claude Code
Sub-agents

Overview

Explain

The orchestrator/sub-agent mental model and why sub-agent-specialization beats one long-context pass

Build

A sub-agent workflow with Claude Code

- Assign roles, spawn via Task, merge through the orchestrator
- Templatize using CLAUDE.md so its a repeatable workflow across an engineering team

Identify

Where sub-agents are the right tool and where they introduce friction (costs, rate-limits, context limits

THE PROBLEM

One pass. Three blind spots.

1

REVIEWER

Asked to hold expertise in logic, testing, and security simultaneously — none of it specialized

3×

DOMAINS

Logic correctness, test coverage, and security vulnerabilities need distinct mental contexts

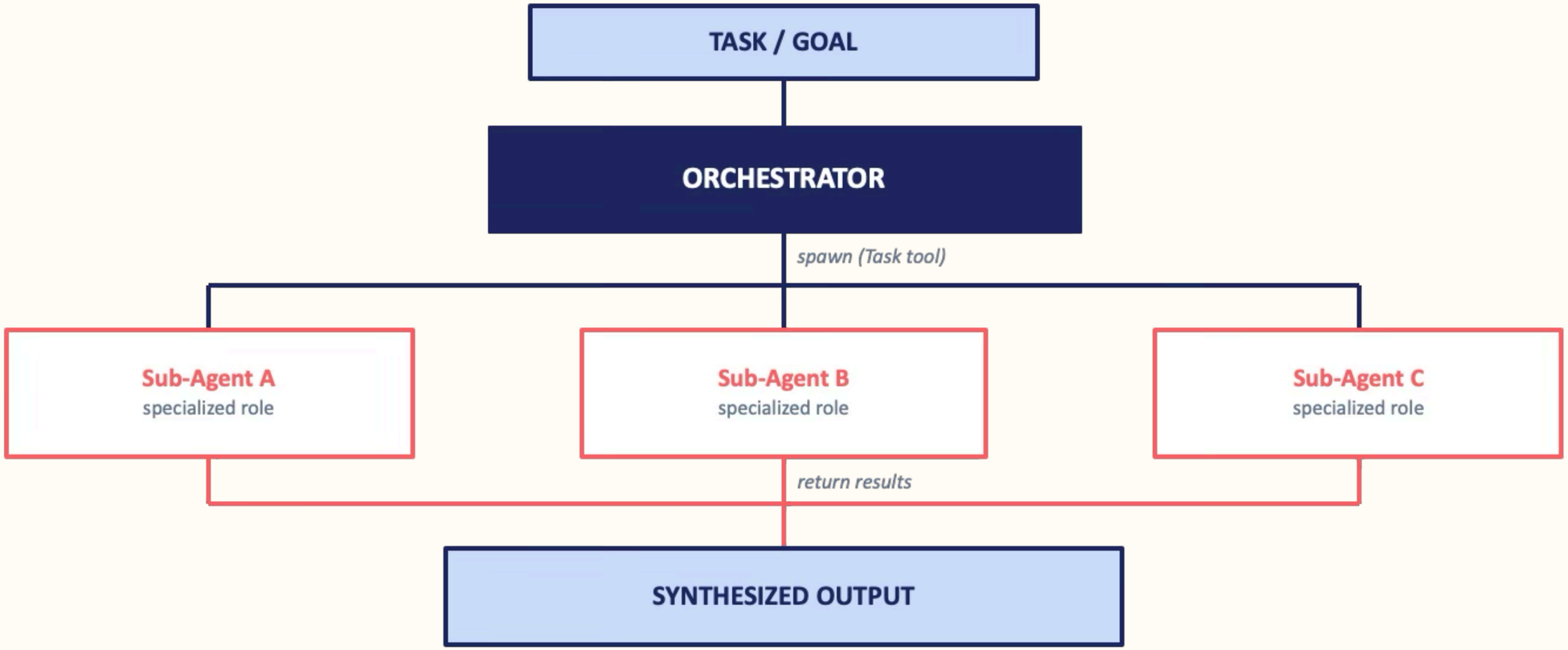
∅

CONSISTENCY

No two reviewers weight the same criteria — results depend on who's available and how tired they are

THE ORCHESTRATOR PATTERN

Decompose. Specialize. Synthesize.



A sub-agent is a specialized Claude instance launched by an orchestrator via the Task tool to execute one focused task ***independently, in parallel with peers***, before returning a structured result.

Specialization versus long context pass

Focus
(role-specific reasoning)

Parallelization
(a 12 min sequential review becomes a 5 min parallel one)

Independent/Task scoped
(Spawned per task, returns result to orchestrator for synthesis)

LIVE DEMO

PR Review, 3 Specialists, Claude.md sub-agent template

CLAUDE.md

From prototype to org-wide standard

One file, automatic pickup

Drop CLAUDE.md at the repo root — Claude Code reads it on every run, with no extra setup required

Zero onboarding required

Every engineer on the org runs the same sub-agent workflow, roles, and synthesis format automatically

Consistent, auditable output

Agent roles, tone, and report structure are locked in — not left to individual prompt variation

CLAUDE.md

```
# PR Review — Sub-Agent Workflow

## Roles
- Logic Reviewer: correctness, edge cases
- Test Writer: coverage gaps, assertions
- Security Auditor: injection, auth, secrets

## Orchestrator instructions
Spawn all three in parallel via Task.
Synthesize into one structured report.
Flag P0 security issues at the top.
```

Where sub-agents fit and where they don't

GOOD FIT

- ✓ Complex multi-step workflows with clear delegation points
- ✓ Tasks requiring parallel execution across domains
- ✓ Processes needing specialized tools or context per step
- ✓ Long-running operations with independent subtasks
- ✓ Workflows where failure isolation matters

NOT A GOOD FIT

- x Simple linear tasks a single prompt can handle
- x Highly interdependent steps needing shared state
- x Low-latency requirements where overhead matters
- x Tasks with ambiguous boundaries between steps
- x Situations where a human-in-the-loop is more effective

Thank you!

Where in your current pipeline would you use a role-specialized sub-agent workflow — and where wouldn't you?